

**ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN**

Tel. (+84.0236) 3736949, Fax. (84-511) 3842771
Website: <http://dut.udn.vn/khoacntt>, E-mail: cntt@dut.udn.vn

BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

**ĐỀ TÀI :
<TÊN ĐỀ TÀI>**

Họ tên sinh viên: Nguyễn Lê Quốc Huy
Mã sinh viên: 102220191
Nhóm HP: 22Nh16

CBHD: TS. Ninh Khánh Duy

Đà Nẵng, 06/2026

MỤC LỤC

MỤC LỤC.....	2
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT.....	4
1.1. Tổng quan về bài toán Text-to-SQL	4
1.1.1. Khái niệm và vai trò.....	4
1.1.2. Các thách thức cốt lõi của bài toán Text-to-SQL	4
1.2. Ứng dụng của Mô hình Ngôn ngữ Lớn (LLM) trong Cơ sở dữ liệu.....	5
1.2.1. Năng lực sinh mã (Code Generation) từ LLM	5
1.2.2. Tối ưu hóa bằng Kỹ thuật Prompt Engineering	6
1.2.3. Ứng dụng LLM trong tối ưu hóa truy vấn (Query Optimization)	6
1.3. Các phương pháp và bộ dữ liệu đánh giá (Evaluation Benchmarks).....	6
1.3.1. Các bộ dữ liệu kiểm thử tiêu chuẩn.....	6
1.3.2. Các chỉ số đánh giá cốt lõi (Evaluation Metrics).....	7
1.4. Kết chương 1	8
CHƯƠNG 2: PHÂN TÍCH THIẾT KẾ HỆ THỐNG.....	9
2.1. PHÁT BIỂU BÀI TOÁN.....	9
2.2. PHÂN TÍCH HIỆN TRẠNG.....	10
2.3. PHÂN TÍCH CHỨC NĂNG	10
2.3.1. Đối tượng sử dụng (Actors).....	10
2.3.2. Phân hệ Sinh và Tối ưu hóa SQL	11
2.3.3. Phân hệ Đánh giá tự động (Evaluation Module - Thành phần cốt lõi)	11
2.3.4. Phân hệ Mô phỏng Cơ sở dữ liệu (Database Simulation / Sandbox)	12
2.3.5. Công nghệ sử dụng và lý do lựa chọn	13
2.4. THIẾT KẾ CƠ SỞ DỮ LIỆU	14
2.4.1. Sơ đồ tổng thể	14
2.4.2. Chi tiết các bảng dữ liệu cốt lõi.....	14
2.5. XÂY DỰNG CHƯƠNG TRÌNH.....	18
2.5.1. Tổ chức cấu trúc thư mục dự án.....	18
2.5.2. Thiết kế Cơ chế Database Simulation (Sandbox Environment).....	19
2.5.3. Quy trình Đánh giá (Evaluation Pipeline Workflow).....	20
2.6. KẾT CHƯƠNG 2.....	22
CHƯƠNG 3: TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ.....	23
3.1. MÔ HÌNH TRIỂN KHAI	23

3.1.1. Môi trường triển khai.....	23
3.1.2. Các công cụ sử dụng.....	23
3.1.3. Cấu hình hệ thống.....	24
3.2. KẾT QUẢ THỰC NGHIỆM.....	25
3.2.1. Kịch bản 1: Sinh truy vấn Text-to-SQL cơ bản	25
3.2.2. Kịch bản 2: Tối ưu hoá truy vấn phức tạp.....	25
3.2.3. Kịch bản 3: Thực thi Batch trên Benchmark.....	26
3.2.4. Kịch bản 4: Trích xuất Báo cáo Đánh giá (Evaluation Report)	26
3.2.5. Kịch bản 5: An toàn thực thi với Database Simulation	27
3.3. NHẬN XÉT ĐÁNH GIÁ KẾT QUẢ	27
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	29
1. KẾT QUẢ ĐẠT ĐƯỢC	29
2. KIẾN NGHỊ VÀ HƯỚNG PHÁT TRIỂN	30
TÀI LIỆU THAM KHẢO	32

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

Chương này trình bày các nội dung cơ sở lý thuyết trọng tâm làm nền tảng cho việc nghiên cứu và phát triển đề án. Nội dung bao gồm sự hình thành và phát triển của bài toán Text-to-SQL, sự can thiệp và hiệu quả của các Mô hình ngôn ngữ lớn (Large Language Models - LLM) vào quy trình tương tác với cơ sở dữ liệu, cũng như các tiêu chuẩn, bộ dữ liệu (benchmarks) đang được giới học thuật sử dụng để đánh giá các mô hình này. Những cơ sở lý thuyết này sẽ được ứng dụng trực tiếp trong phần phân tích, thiết kế và triển khai hệ thống SQLTuner ở các chương tiếp theo.

1.1. Tổng quan về bài toán Text-to-SQL

1.1.1. Khái niệm và vai trò

Text-to-SQL (chuyển đổi văn bản thành câu lệnh SQL) là một bài toán quan trọng thuộc lĩnh vực Xử lý Ngôn ngữ Tự nhiên (NLP) kết hợp với Cơ sở dữ liệu (Database). Mục tiêu của bài toán là tự động hóa việc dịch các câu hỏi, yêu cầu bằng ngôn ngữ tự nhiên (Natural Language) của người dùng thành các câu lệnh truy vấn SQL (Structured Query Language) hợp lệ và có thể thực thi được trên các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS).

Quá trình này đóng vai trò như một cầu nối, giúp xóa bỏ rào cản kỹ thuật. Thay vì phải am hiểu về cú pháp SQL, cấu trúc bảng (schema), và các hàm thao tác dữ liệu phức tạp, người dùng đầu cuối (nhân viên nghiệp vụ, quản lý) chỉ cần đặt câu hỏi bằng ngôn ngữ thông thường (ví dụ: *"Cho tôi biết top 5 nhân viên có doanh số cao nhất quý 1 năm 2026"*).

1.1.2. Các thách thức cốt lõi của bài toán Text-to-SQL

Mặc dù mang lại giá trị thực tiễn cao, Text-to-SQL là một bài toán đầy thách thức do sự khác biệt lớn giữa tính linh hoạt của ngôn ngữ tự nhiên và tính quy tắc nghiêm ngặt của ngôn ngữ lập trình. Các thách thức chính bao gồm:

- **Sự nhập nhằng trong ngôn ngữ tự nhiên (Ambiguity):** Một câu hỏi có thể được diễn đạt theo nhiều cách khác nhau, hoặc thiếu đi các từ khóa rõ ràng để ánh xạ trực tiếp với tên bảng/cột trong cơ sở dữ liệu.
- **Liên kết lược đồ (Schema Linking):** Khả năng hệ thống nhận diện đúng các thực thể được nhắc đến trong câu hỏi và ánh xạ chúng chính xác vào các bảng (tables), cột (columns), và khóa ngoại (foreign keys) tương ứng trong cơ sở dữ liệu. Đây là bước quyết định sự thành bại của câu truy vấn.
- **Cấu trúc truy vấn phức tạp (Complex Query Structures):** Đối với các cơ sở dữ liệu lớn, việc trích xuất thông tin thường đòi hỏi các thao tác lồng nhau (Nested Queries), kết nối nhiều bảng (Multiple JOINS), nhóm dữ liệu (GROUP BY), và lọc điều kiện phức tạp (HAVING, mệnh đề điều kiện lồng).
- **Khả năng tổng quát hóa chéo miền (Cross-domain Generalization):** Một hệ thống Text-to-SQL tốt phải có khả năng hoạt động trên các cơ sở dữ liệu mới (zero-shot) mà nó chưa từng được huấn luyện trước đó.

1.2. Ứng dụng của Mô hình Ngôn ngữ Lớn (LLM) trong Cơ sở dữ liệu

Sự bùng nổ của các Mô hình Ngôn ngữ Lớn (LLMs) dựa trên kiến trúc Transformer đã tạo ra một bước ngoặt lớn trong cách giải quyết bài toán Text-to-SQL và quản trị cơ sở dữ liệu.

1.2.1. Năng lực sinh mã (Code Generation) từ LLM

Các LLM hiện đại được huấn luyện trên một lượng dữ liệu khổng lồ bao gồm cả mã nguồn và văn bản. Do đó, chúng sở hữu khả năng suy luận logic và sinh mã lập trình (code generation) vượt trội. Đối với SQL, thay vì phải xây dựng các quy tắc phân tích cú pháp (parser) thủ công như các phương pháp truyền thống, LLM có thể học được mối quan hệ giữa câu hỏi tự nhiên và cú pháp SQL tương ứng.

1.2.2. Tối ưu hóa bằng Kỹ thuật Prompt Engineering

Để LLM sinh ra câu lệnh SQL chính xác với một cơ sở dữ liệu cụ thể, kỹ thuật Prompt Engineering được áp dụng mạnh mẽ. Thông qua quá trình thiết kế Prompt, hệ thống sẽ cung cấp cho LLM:

- **Siêu dữ liệu (Metadata):** Bao gồm ngôn ngữ định nghĩa dữ liệu (DDL), tên bảng, tên cột, kiểu dữ liệu và mô tả khóa chính/khóa ngoại.
- **Dữ liệu mẫu (Few-shot prompting):** Đưa ra một vài cặp ví dụ (câu hỏi tự nhiên - câu lệnh SQL) để hướng dẫn mô hình học theo định dạng mong muốn.
- **Suy luận từng bước (Chain-of-Thought):** Yêu cầu mô hình giải thích logic từng bước trước khi viết SQL, giúp tăng cường khả năng xử lý các yêu cầu phức tạp.

1.2.3. Ứng dụng LLM trong tối ưu hóa truy vấn (Query Optimization)

Bên cạnh việc sinh truy vấn, LLM còn được ứng dụng để tối ưu hóa hiệu năng cơ sở dữ liệu. Bằng cách cung cấp thông tin từ Kế hoạch thực thi (Execution Plan) thu được qua các lệnh như EXPLAIN hoặc EXPLAIN ANALYZE, LLM có khả năng phân tích các điểm nghẽn (bottlenecks) – ví dụ như thao tác quét toàn bảng (Sequential Scan) tốn kém. Từ đó, AI có thể đề xuất các phương án tối ưu như: viết lại cấu trúc mệnh đề JOIN, thêm các chỉ mục (Indexes) phù hợp, hoặc thay đổi cách thức lọc dữ liệu nhằm giảm chi phí tài nguyên và thời gian phản hồi.

1.3. Các phương pháp và bộ dữ liệu đánh giá (Evaluation Benchmarks)

Để đảm bảo tính khách quan và đo lường chính xác hiệu năng của các hệ thống và chiến lược Text-to-SQL, việc sử dụng các bộ dữ liệu tiêu chuẩn (Benchmarks) là bắt buộc. Đồ án tập trung nghiên cứu và sử dụng hai bộ dữ liệu học thuật hàng đầu hiện nay.

1.3.1. Các bộ dữ liệu kiểm thử tiêu chuẩn

- **Spider Benchmark:**

- *Đặc điểm:* Là một trong những bộ dữ liệu đánh giá Text-to-SQL quy mô lớn và phổ biến nhất. Spider nổi bật với đặc tính đa miền (cross-domain), chứa hàng ngàn cơ sở dữ liệu thuộc nhiều lĩnh vực khác nhau.
- *Thử thách:* Nó đặt ra môi trường thử nghiệm zero-shot nghiêm ngặt (mô hình không được thấy cơ sở dữ liệu test trong quá trình huấn luyện) và phân loại độ khó của các truy vấn theo 4 mức độ: Dễ (Easy), Trung bình (Medium), Khó (Hard), và Cực khó (Extra Hard). Các câu lệnh ở mức độ khó thường yêu cầu nhiều phép JOIN, truy vấn con lồng nhau và các phép toán tập hợp.
- **BIRD Benchmark (A Big Bench for Large-Scale Database Grounded Text-to-SQLs):**
 - *Đặc điểm:* Đây là bộ dữ liệu thế hệ mới, được thiết kế để lấp đầy khoảng trống giữa môi trường học thuật và thực tiễn công nghiệp.
 - *Thử thách:* BIRD chú trọng vào các cơ sở dữ liệu có quy mô rất lớn (large-scale), dữ liệu bị nhiễu (messy data) và đòi hỏi hệ thống phải hiểu sâu về ngữ nghĩa dữ liệu bên trong (database-grounded). Hơn nữa, để giải quyết các câu hỏi trong BIRD, mô hình thường cần kết hợp với các kiến thức ngoại vi (external knowledge base) – mô phỏng chính xác cách một kỹ sư phân tích dữ liệu làm việc thực tế.

1.3.2. Các chỉ số đánh giá cốt lõi (Evaluation Metrics)

Kết quả mô hình không thể chỉ được đánh giá bằng mắt thường mà cần đo lường bằng các công thức toán học chặt chẽ. Hệ thống sử dụng hai chỉ số chính:

- **Execution Accuracy (EX - Độ chính xác thực thi):**
 - *Định nghĩa:* Đây là tiêu chuẩn vàng (gold standard) trong đánh giá Text-to-SQL. Hệ thống sẽ thực thi cả câu lệnh truy vấn do AI sinh ra

(Predicted SQL) và câu lệnh truy vấn đáp án (Ground Truth SQL) trên cùng một trạng thái cơ sở dữ liệu thực.

- *Đánh giá:* Nếu tập kết quả (Result Set) trả về từ hai câu lệnh này hoàn toàn khớp nhau, truy vấn đó được đánh giá là chính xác (Pass). Tiêu chí này rất công bằng và thực tế vì một yêu cầu nghiệp vụ hoàn toàn có thể được giải quyết bằng nhiều cú pháp SQL khác nhau.

- **Exact Match (EM - Độ khớp chính xác):**

- *Định nghĩa:* Đánh giá mức độ trùng khớp tuyệt đối về mặt hình thức (cú pháp) giữa câu lệnh dự đoán và câu lệnh chuẩn.
- *Đánh giá:* Trước khi so sánh, cả hai câu lệnh đều trải qua quá trình chuẩn hóa (loại bỏ khoảng trắng thừa, đồng bộ hóa chữ hoa/chữ thường, sắp xếp lại thứ tự các điều kiện không phụ thuộc logic). Chỉ số này giúp kiểm tra khả năng bắt chước cấu trúc nghiêm ngặt của mô hình, tuy nhiên thường có điểm số thấp hơn EX do không tính đến sự đa dạng trong cách viết truy vấn.

1.4. Kết chương 1

Chương 1 đã cung cấp một cái nhìn tổng quan và toàn diện về cơ sở lý thuyết của bài toán Text-to-SQL. Đi từ việc định nghĩa, nhận diện các thách thức cốt lõi, cho đến việc phân tích tiềm năng to lớn của Mô hình Ngôn ngữ Lớn (LLM) trong việc tự động hóa và tối ưu hóa truy vấn cơ sở dữ liệu. Đồng thời, chương cũng thiết lập các tiêu chuẩn khắt khe về phương pháp đo lường thông qua các benchmarks uy tín (Spider, BIRD) và các chỉ số (EX, EM). Hệ thống lý thuyết vững chắc này chính là tiền đề quan trọng để làm cơ sở biện luận cho các giải pháp kiến trúc, phân tích và thiết kế hệ thống SQLTuner sẽ được trình bày chi tiết trong Chương 2.

CHƯƠNG 2: PHÂN TÍCH THIẾT KẾ HỆ THỐNG

2.1. PHÁT BIỂU BÀI TOÁN

Trong kỷ nguyên số hóa và quản trị dữ liệu lớn, ngôn ngữ truy vấn cấu trúc (SQL) đóng vai trò tối quan trọng trong việc khai thác thông tin từ các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS), đặc biệt là PostgreSQL. Tuy nhiên, việc xây dựng, kiểm thử và tối ưu hóa các câu lệnh SQL phức tạp thường đòi hỏi các kỹ sư dữ liệu (Data Engineers) và lập trình viên Backend phải tiêu tốn rất nhiều thời gian cũng như kinh nghiệm chuyên môn sâu sắc về hạ tầng lưu trữ.

Sự trỗi dậy của các Mô hình Ngôn ngữ Lớn (LLMs) mở ra hướng đi mới cho bài toán tự động hóa sinh mã SQL (Text-to-SQL). Mặc dù vậy, các giải pháp hiện tại trên thị trường vẫn còn mang tính chất rời rạc và bộc lộ ba điểm nghẽn lớn:

1. **Thiếu tính tối ưu hiệu năng:** Các công cụ đa phần chỉ dừng lại ở việc sinh cú pháp SQL hợp lệ về mặt ngữ nghĩa (Semantics) mà chưa có khả năng đánh giá hoặc tinh chỉnh để câu lệnh đạt hiệu năng thực thi tốt nhất trên môi trường thực tế.
2. **Thiếu cơ chế kiểm định tự động:** Chưa có một bộ khung (framework) tiêu chuẩn tích hợp cho phép kiểm thử và đánh giá hàng loạt (batch evaluation) chất lượng của các câu lệnh do AI sinh ra dựa trên các tập dữ liệu mẫu (benchmarks).
3. **Khó kiểm soát lỗi hồi quy (Regression):** Khi lập trình viên thay đổi cấu trúc Prompt (Prompt Engineering) hoặc thay đổi mô hình LLM nền tảng, hệ thống không có công cụ đo lường định lượng để biết được độ chính xác tổng thể tăng hay giảm.

Yêu cầu đặt ra là phải xây dựng hệ thống **SQLTuner** – một nền tảng toàn diện tích hợp chặt chẽ cả ba quá trình cốt lõi bao gồm: **Sinh truy vấn thông minh**, **Tối ưu hóa hiệu năng dựa trên kế hoạch thực thi thực tế**, và **Đánh giá tự động hàng loạt** để quản lý vòng đời chất lượng mô hình.

2.2. PHÂN TÍCH HIỆN TRẠNG

Khảo sát thực tế các công cụ hỗ trợ lập trình hiện nay (như GitHub Copilot, ChatGPT, hay các extension hỗ trợ AI trên các IDE) cho thấy các công cụ này hỗ trợ nhà phát triển tương đối tốt ở mức độ sinh mã đơn lẻ (single-turn query creation). Tuy nhiên, đối với quy trình Kỹ nghệ Dữ liệu (Data Engineering) chuyên nghiệp, các giải pháp này bộc lộ những hạn chế rõ rệt:

- **Sự thiếu kết nối với trạng thái Database:** Các AI assistant thông thường không thể nhận biết được dung lượng bảng, sự hiện diện của các Index, hay phân phối dữ liệu (data distribution) thực tế trong database. Do đó, câu lệnh sinh ra dễ rơi vào tình trạng chạy quá thời gian (Timeout) hoặc quét toàn bộ bảng (Sequential Scan) khi đối mặt với hàng triệu bản ghi.
- **Khâu kiểm định độ chính xác bị bỏ ngỏ:** Các kỹ sư phải sao chép thủ công câu lệnh SQL từ cửa sổ chat của AI, dán vào database client để chạy thử và kiểm tra tập kết quả trả về. Quy trình thủ công này không thể áp dụng khi cần kiểm định độ chính xác của mô hình trên quy mô hàng ngàn câu hỏi thuộc các miền tri thức khác nhau trước khi đưa hệ thống vào vận hành chính thức (Production).

2.3. PHÂN TÍCH CHỨC NĂNG

2.3.1. Đối tượng sử dụng (Actors)

Hệ thống SQLTuner hướng tới hai đối tượng người dùng chính với các mục tiêu tương ứng:

- **Lập trình viên Backend / Quản trị viên cơ sở dữ liệu (DBA):** Nhóm đối tượng này sử dụng hệ thống để chuyển đổi nhanh các yêu cầu nghiệp vụ thành mã SQL, đồng thời tận dụng module tối ưu hóa để phân tích các câu lệnh chậm, từ đó nhận các đề xuất cải thiện hiệu năng (như tái cấu trúc câu lệnh JOIN hoặc tạo chỉ mục bổ sung).

- **Nhà nghiên cứu AI / Kỹ sư dữ liệu (AI Researchers / Data Engineers):** Nhóm đối tượng này sử dụng phân hệ Đánh giá (Evaluation Framework) như một công cụ chuyên sâu để đo lường, chấm điểm định lượng hiệu suất của các mô hình LLM khác nhau hoặc đánh giá các kỹ thuật Prompt Engineering mới dựa trên các tập dữ liệu chuẩn.

2.3.2. Phân hệ Sinh và Tối ưu hóa SQL

- **Sinh truy vấn (Text-to-SQL Generation):** Hệ thống trích xuất thông tin lược đồ cơ sở dữ liệu (Database Schema) bao gồm tên bảng, cột, kiểu dữ liệu và các ràng buộc khóa chính/khóa ngoại. Thông tin này được cấu trúc hóa tự động thành một ngữ cảnh prompt (Context Prompt) gửi tới LLM cùng với câu hỏi ngôn ngữ tự nhiên của người dùng để thu về câu lệnh SQL tương ứng.
- **Tối ưu hóa truy vấn (SQL Optimization):** Hệ thống tiếp nhận câu lệnh SQL cần tối ưu, thực thi ngầm lệnh EXPLAIN (ANALYZE, FORMAT JSON) trên hệ quản trị cơ sở dữ liệu để lấy về cây kế hoạch thực thi (Execution Plan) chi tiết. Phân hệ Optimizer sẽ bóc tách các chỉ số về chi phí tính toán (Cost), thời gian thực hiện (Actual Time), và các thao tác thô bạo trên bộ nhớ. Dựa vào đó, module AI sẽ đưa ra đề xuất câu lệnh mới cụ thể nhằm tối ưu hóa tài nguyên.

2.3.3. Phân hệ Đánh giá tự động (Evaluation Module - Thành phần cốt lõi)

Phân hệ này hoạt động như một công cụ kiểm thử tự động (Automated Testing Framework) chạy ở chế độ nền hoặc kích hoạt qua giao diện:

- **Quản lý và Nạp Dataset Benchmark:** Hỗ trợ import các tập dữ liệu cấu trúc chuẩn hóa như Spider và BIRD, bao gồm danh sách các câu hỏi ngôn ngữ tự nhiên, cơ sở dữ liệu đích tương ứng, và câu lệnh SQL đáp án chuẩn (Ground Truth).

- **Thực thi suy luận hàng loạt (Batch Inference):** Tự động phân bổ toàn bộ danh sách câu hỏi trong tập dữ liệu (dataset) để xử lý qua các mô hình ngôn ngữ được triển khai cục bộ (Local LLM) thông qua cơ chế bất đồng bộ. Hệ thống được tích hợp thuật toán điều phối hàng đợi (Queue Management) và kiểm soát kích thước lô xử lý (Batch Size) nhằm tối ưu hóa hiệu năng phần cứng (CPU/GPU) và ngăn chặn tình trạng tràn bộ nhớ (Out of Memory - OOM) trong suốt quá trình thực thi.
- **Công cụ chấm điểm và phân tích lỗi (Evaluation Engine):** Chạy song song hai câu lệnh SQL (Predicted SQL và Ground Truth SQL) trong một môi trường cô lập bảo mật (Sandbox), so sánh dữ liệu kết quả trả về để tính toán chính xác hai chỉ số điểm số **Execution Accuracy (EX)** và **Exact Match (EM)**. Các bản ghi lỗi cú pháp (Syntax Error) hoặc lỗi thực thi (Runtime Error) sẽ được bắt lại chi tiết cùng với thông báo lỗi từ hệ thống cơ sở dữ liệu để phục vụ công tác gỡ lỗi (Debugging).

2.3.4. Phân hệ Mô phỏng Cơ sở dữ liệu (Database Simulation / Sandbox)

Để đảm bảo an toàn tuyệt đối khi thực thi tự động mã nguồn do AI sinh ra, hệ thống yêu cầu một phân hệ môi trường mô phỏng (Sandbox). Phân hệ này chịu trách nhiệm:

- **Cách ly thực thi (Execution Isolation):** Tạo ra các lược đồ ảo (temporary schemas) hoặc sử dụng cơ chế Transaction Rollback để chạy thử các câu lệnh Predicted SQL. Điều này ngăn chặn triệt để các rủi ro mất mát dữ liệu khi LLM sinh sai cú pháp chứa các lệnh phá hủy (như DROP, TRUNCATE, UPDATE không có mệnh đề WHERE).
- **Mô phỏng trạng thái dữ liệu (Data Mocking):** Hỗ trợ nạp nhanh một tập dữ liệu con (subset data) mang tính đại diện từ cơ sở dữ liệu gốc vào môi trường Sandbox, giúp thao tác so khớp tập kết quả (Result Match) diễn ra chính xác mà không tốn chi phí quét toàn bộ dữ liệu lớn thực tế.

- **Quản lý vòng đời (Lifecycle Management):** Tự động dọn dẹp (Teardown) và thu hồi tài nguyên bộ nhớ ngay sau khi kết thúc đối chiếu kết quả của một câu hỏi, trả lại trạng thái nguyên bản sạch sẽ cho lượt đánh giá tiếp theo.

2.3.5. Công nghệ sử dụng và lý do lựa chọn

Hệ thống lựa chọn các công nghệ hiện đại nhằm đảm bảo tính phân tách mã nguồn và hiệu năng vận hành cao:

- **Backend (Python & FastAPI):** Python là ngôn ngữ tiêu chuẩn cho các tác vụ làm việc với AI và xử lý dữ liệu. FastAPI được lựa chọn nhờ cơ chế ASGI hỗ trợ lập trình bất đồng bộ (async/await) mạnh mẽ, giúp tối ưu hóa hiệu năng khi hệ thống phải thực hiện hàng ngàn yêu cầu I/O-bound đồng thời (gọi LLM và truy vấn Database liên tục trong quá trình Evaluation).
- **Cơ sở dữ liệu (PostgreSQL):** Hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, mã nguồn mở, hỗ trợ các tính năng phân tích kế hoạch thực thi sâu sắc phục vụ hoàn hảo cho module tối ưu hóa.
- **Frontend (React, Vite, TypeScript):** Xây dựng giao diện Single Page Application (SPA) mượt mà. Vite giúp tăng tốc độ build và hot-reload trong quá trình phát triển; TypeScript đảm bảo tính chặt chẽ của kiểu dữ liệu, giảm thiểu lỗi runtime ở phía giao diện người dùng khi tương tác với các cấu trúc log phức tạp.
- **Triển khai (Docker & Google Cloud Run):** Toàn bộ ứng dụng được đóng gói thành các Docker Image độc lập, giúp loại bỏ xung đột môi trường. Triển khai trên Google Cloud Run giúp hệ thống tự động co giãn (Auto-scaling) theo cơ chế Serverless, tối ưu chi phí hạ tầng khi chỉ tính toán tiền dựa trên thời gian chạy các batch evaluation lớn.

2.4. THIẾT KẾ CƠ SỞ DỮ LIỆU

2.4.1. Sơ đồ tổng thể

Cơ sở dữ liệu của hệ thống SQLTuner được thiết kế theo mô hình quan hệ (Relational Database) nhằm quản lý toàn diện luồng dữ liệu từ người dùng, cấu hình hệ thống cho đến kết quả phân tích AI. Sơ đồ tổng thể bao gồm các nhóm dữ liệu chính:

- **Nhóm định danh và phiên:** Quản lý người dùng (users) và các phiên đăng nhập (user_sessions).
- **Nhóm cấu hình kết nối:** Lưu trữ thông tin và siêu dữ liệu (metadata) của các cơ sở dữ liệu đích (db_connections).
- **Nhóm tương tác và sinh truy vấn:** Ghi nhận các đoạn hội thoại (conversations) và chi tiết từng lượt suy luận ngôn ngữ tự nhiên thành câu lệnh SQL (query_logs).
- **Nhóm tối ưu và phản hồi:** Chứa dữ liệu bóc tách từ kế hoạch thực thi để đưa ra đề xuất tối ưu (performance_analysis) và các đánh giá, hiệu chỉnh từ người dùng (feedbacks).

Sự liên kết chặt chẽ giữa các bảng này giúp hệ thống truy xuất nhanh chóng ngữ cảnh của cơ sở dữ liệu để cung cấp cho Local LLM, đồng thời lưu vết toàn bộ lịch sử để phục vụ cho việc đánh giá và tinh chỉnh mô hình sau này.

2.4.2. Chi tiết các bảng dữ liệu cốt lõi

Bảng db_connections (Lưu trữ thông tin kết nối và siêu dữ liệu cơ sở dữ liệu đích)

Bảng này đóng vai trò quan trọng trong việc cung cấp ngữ cảnh (Schema) cho LLM. Nó không chỉ lưu thông tin đăng nhập mà còn cache lại cấu trúc bảng của database đích.

Tên trường (Column)	Kiểu dữ liệu (Type)	Ràng buộc (Constraint)	Mô tả (Description)
id	UUID	Primary Key	Khóa chính tự sinh (uuid_generate_v4)
user_id	UUID	Foreign Key -> users(id)	Liên kết với tài khoản người dùng
name	VARCHAR	Not Null	Tên gọi nhớ của kết nối
host, port	VARCHAR, INT		Địa chỉ máy chủ và cổng (Mặc định 5432)
db_type	db_type	Default 'postgres'	Phân loại hệ quản trị CSDL
metadata_cache	JSONB	Nullable	Lưu trữ cache siêu dữ liệu của database
meta_schema	JSONB	Default '{}'	Lưu trữ cấu trúc lược đồ (DDL) đã được trích xuất

Bảng query_logs (Lưu vết chi tiết quá trình sinh truy vấn Text-to-SQL)

Bảng này ghi nhận mọi tương tác giữa người dùng và mô hình AI nội bộ, phục vụ trực tiếp cho việc theo dõi chất lượng sinh truy vấn.

Tên trường (Column)	Kiểu dữ liệu (Type)	Ràng buộc (Constraint)	Mô tả (Description)
id	UUID	Primary Key	Khóa chính của lượt tương tác
conversation_id	UUID	Foreign Key	Thuộc phiên hội thoại nào
role	USER-DEFINED	Not Null	Vai trò (user, system, hoặc assistant)
content	TEXT	Not Null	Nội dung câu hỏi (ngôn ngữ tự nhiên)
sql_generated	TEXT	Nullable	Câu lệnh SQL do Local LLM sinh ra
action_type	VARCHAR	Default 'chat'	Phân loại hành động
schema_generated	JSONB	Nullable	Lưu vết schema được dùng làm prompt ngữ cảnh

Bảng performance_analysis (Lưu trữ dữ liệu phân tích và đề xuất tối ưu hiệu năng)

Lưu lại kết quả sau khi hệ thống thực thi câu lệnh EXPLAIN và được LLM phân tích.

Tên trường (Column)	Kiểu dữ liệu (Type)	Ràng buộc (Constraint)	Mô tả (Description)
id	UUID	Primary Key	Khóa chính tự sinh
query_log_id	UUID	Foreign Key, Unique	Liên kết 1-1 với lượt sinh SQL tương ứng
execution_time_ms	DOUBLE PRECISION	Nullable	Thời gian thực thi thực tế (ms)
total_cost	DOUBLE PRECISION	Nullable	Chi phí tính toán ước lượng từ CSDL
explain_plan	JSONB	Not Null	Cây kế hoạch thực thi chi tiết dạng JSON
index_recommendation	TEXT	Nullable	Đề xuất tối ưu (như thêm Index) do AI tạo ra

2.5. XÂY DỰNG CHƯƠNG TRÌNH

2.5.1. Tổ chức cấu trúc thư mục dự án

Mã nguồn của hệ thống được tổ chức theo kiến trúc phân tầng dạng module hóa rõ ràng, giúp cô lập phạm vi nghiệp vụ và tối ưu hóa việc quản lý tài nguyên khi làm việc với LLM:

Plaintext

sqltuner/

```
|— backend/          # Mã nguồn dịch vụ Backend (FastAPI)
| | — app/
| | | — api/         # Các RESTful API Endpoints (v1)
| | | | — router.py  # Định tuyến tổng hợp
| | | | — generate.py # API phục vụ sinh truy vấn đơn lẻ
| | | | — optimize.py # API thực thi EXPLAIN và trả về đề xuất tối ưu
| | | | — evaluation.py # API quản lý và kích hoạt đợt chạy test hàng loạt
| | | — core/        # Khởi tạo hệ thống (Config, Security, DB Connection)
| | | — models/       # Khái niệm thực thể cơ sở dữ liệu (ORM/SQLAlchemy)
| | | — schemas/      # Định nghĩa dữ liệu đầu vào/đầu ra (Pydantic)
| | | — services/     # Tầng logic nghiệp vụ cốt lõi
| | | — llm_client.py # Trình giao tiếp và quản lý tài nguyên của Local LLM
| | | — optimizer.py  # Logic bóc tách cây kế hoạch và tối ưu câu lệnh
| | | — evaluator.py  # Động cơ điều phối tiến trình chạy Evaluation
| | — requirements.txt # Danh sách thư viện phụ thuộc của Python
| — Dockerfile       # Cấu hình đóng gói container cho Backend
```

```

└── frontend/          # Giao diện người dùng (React SPA)
    └── src/
        ├── components/  # Các UI Component dùng chung (Table, Terminal,
CodeEditor)
        ├── pages/       # Các trang giao diện chính
        ├── Dashboard.tsx# Trang chủ hiển thị biểu đồ phân tích
        ├── Optimizer.tsx# Không gian làm việc tối ưu hóa truy vấn
        ├── Evaluation.tsx# Giao diện quản lý Benchmark và cấu hình LLM
        └── services/     # Client gọi API kết nối sang FastAPI
    ├── package.json     # Quản lý thư viện JS/TS
    └── vite.config.ts    # Cấu hình đóng gói mã nguồn của Vite
└── docker-compose.yml   # Tập cấu hình điều phối chạy toàn bộ hệ thống

```

2.5.2. Thiết kế Cơ chế Database Simulation (Sandbox Environment)

Thành phần Database Simulation được xây dựng sâu bên trong tầng Service của Backend FastAPI nhằm quản lý vòng đời thực thi truy vấn một cách an toàn và tối ưu tài nguyên. Kiến trúc của cơ chế này được thiết kế theo mô hình **Transaction-based Sandbox**:

- **Khởi tạo kết nối cách ly (Isolated Session Init):** Khi tiến trình đánh giá bắt đầu, hệ thống sẽ không sử dụng chung Connection Pool của ứng dụng chính. Thay vào đó, một phiên kết nối (Session) độc lập với PostgreSQL được khởi tạo thông qua thư viện SQLAlchemy.
- **Cơ chế bảo vệ bằng Transaction & Savepoint:** Trước khi thực thi bất kỳ câu lệnh SQL nào do AI sinh ra, hệ thống tự động mở một Database Transaction (BEGIN) và đánh dấu một điểm lưu (SAVEPOINT).

- **Thực thi và Cảnh báo (Execution & Catching):** Câu lệnh Predicted SQL được chạy trong vùng an toàn này. Nếu hệ thống phát hiện các truy vấn DML/DDL làm thay đổi trạng thái dữ liệu thay vì chỉ truy vấn (SELECT), nó sẽ bị bắt lại thông qua luồng ngoại lệ (Exception Handling).
- **Thu hồi trạng thái (Rollback & Teardown):** Bất kể truy vấn thành công trả về tập dữ liệu (Result Set) hay thất bại (Runtime/Syntax Error), hệ thống đều gọi lệnh ROLLBACK để hủy bỏ mọi thay đổi tạm thời, đảm bảo cơ sở dữ liệu kiểm thử luôn giữ nguyên trạng thái gốc (Zero-mutation) cho hàng ngàn câu lệnh phía sau.

2.5.3. Quy trình Đánh giá (Evaluation Pipeline Workflow)

Quy trình thực thi của Phân hệ Đánh giá tự động được thiết kế thành một đường ống xử lý tuần tự nghiêm ngặt, đảm bảo tính chính xác cao và quản lý chặt chẽ tài nguyên phần cứng nội bộ:

- **Bước 1: Tiếp nhận và Khởi tạo (Initialization):** Người dùng chọn tập dữ liệu (ví dụ: Spider) và cấu hình thông số mô hình cục bộ cần đánh giá trên giao diện Frontend. Backend tiến hành đọc danh sách dữ liệu từ tệp cấu hình của benchmark, tạo bản ghi trạng thái chờ xử lý.
- **Bước 2: Xây dựng Prompt đồng bộ (Schema Linking):** Với mỗi câu hỏi trong hàng đợi, hệ thống tự động trích xuất thông tin meta_schema (DDL) từ bảng db_connections của cơ sở dữ liệu đích tương ứng để gắn vào Prompt mẫu, đảm bảo mô hình nhận đủ thông tin về các trường và bảng.
- **Bước 3: Suy luận bất đồng bộ trên LLM (Asynchronous Inference):** Tiến trình gọi dịch vụ suy luận được thực hiện bất đồng bộ tới các mô hình ngôn ngữ lớn được triển khai cục bộ (Local LLM) trên máy chủ nội bộ. Để đảm bảo tính ổn định, hệ thống áp dụng cơ chế quản lý hàng đợi (Queue Management) và kiểm soát nghiêm ngặt kích thước lô xử lý (Batch Size). Việc này giúp điều phối tối ưu tài nguyên tính toán (CPU/GPU) và ngăn chặn

triệt để tình trạng tràn bộ nhớ VRAM (Out of Memory - OOM) trong quá trình suy luận cường độ cao. Câu lệnh SQL sinh ra được lưu tạm thời vào bộ nhớ làm việc.

- **Bước 4: Thực thi an toàn trong Database Simulation (Execution Sandbox):**

- Hệ thống kích hoạt module Sandbox, mở một Transaction cô lập trên cơ sở dữ liệu PostgreSQL.
- Thực thi câu lệnh SQL vừa sinh ra (Predicted SQL) để lấy tập dữ liệu kết quả $SR_{\{pred\}}$. Nếu có lỗi cú pháp, ghi nhận trạng thái lỗi và gán điểm số $EX = 0$.
- Thực thi câu lệnh SQL đáp án (Ground Truth SQL) để lấy tập dữ liệu chuẩn $SR_{\{true\}}$.
- Sử dụng thuật toán so sánh sâu (Deep Comparison) giữa $SR_{\{pred\}}$ và $SR_{\{true\}}$. Nếu trùng khớp hoàn toàn, gán điểm $EX = 1$, ngược lại gán $EX = 0$.

- **Bước 5: Chuẩn hóa và so khớp cú pháp (Exact Match Checking):** Hệ thống chuẩn hóa chuỗi văn bản của cả hai câu lệnh SQL (chuyển về chữ thường, loại bỏ khoảng trắng thừa, đồng bộ hóa thứ tự viết tên bảng) và thực hiện so sánh chuỗi trực tiếp để đưa ra điểm số EMS (0 hoặc 1).

- **Bước 6: Ghi nhật ký và Hoàn tất (Logging & Finalization):** Kết quả của từng câu hỏi được lưu chi tiết vào cơ sở dữ liệu (query_logs). Sau khi duyệt qua toàn bộ danh sách, hệ thống tính toán điểm số trung bình cộng tổng thể, cập nhật trạng thái đợt kiểm thử thành hoàn tất và gửi tín hiệu cho Frontend để hiển thị biểu đồ phân tích độ chính xác.

2.6. KẾT CHƯƠNG 2

Chương 2 đã hoàn thành việc mô tả chi tiết bức tranh toàn cảnh về mặt phân tích và thiết kế hệ thống của dự án SQLTuner. Bắt đầu từ việc định vị rõ ràng các điểm nghẽn thực tế của bài toán Text-to-SQL và việc tích hợp AI vào quy trình kỹ nghệ dữ liệu, chương đã vạch rõ các phân hệ chức năng tương ứng với từng đối tượng sử dụng cụ thể. Đồng thời, thiết kế cơ sở dữ liệu thực tế dựa trên mô hình quan hệ và sơ đồ tổ chức cấu trúc thư mục module hóa đã được thiết lập chặt chẽ. Trọng tâm cốt lõi của chương là việc làm rõ kiến trúc đường ống xử lý (Pipeline Workflow) tận dụng hoàn toàn sức mạnh của các Local LLM, đảm bảo tính bảo mật dữ liệu và tự chủ về hạ tầng. Đây chính là bản thiết kế chi tiết tạo tiền đề vững chắc cho việc hiện thực hóa mã nguồn, triển khai thực nghiệm và đánh giá số liệu thực tế sẽ được trình bày tại Chương 3.

CHƯƠNG 3: TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ

Chương này trình bày chi tiết về kết quả triển khai hệ thống SQLTuner trong môi trường thực tế, bao gồm cách thức cấu hình hệ thống, công cụ sử dụng và việc tích hợp các mô hình ngôn ngữ lớn nội bộ (Local LLM). Kết quả của hệ thống được đánh giá khách quan và toàn diện thông qua các kịch bản thực nghiệm từ cơ bản đến phức tạp nhằm minh chứng cho tính hiệu quả, độ ổn định của các module chức năng, đặc biệt là bộ khung Đánh giá (Evaluation Framework) đã được đề xuất ở chương trước.

3.1. MÔ HÌNH TRIỂN KHAI

3.1.1. Môi trường triển khai

Để đảm bảo tính độc lập, dễ dàng bảo trì và đồng bộ hóa môi trường vận hành, toàn bộ hệ thống SQLTuner (bao gồm Backend, Frontend và cơ sở dữ liệu) được đóng gói (container hóa) bằng nền tảng **Docker**.

- **Tính nhất quán:** Việc sử dụng docker-compose giúp hệ thống thiết lập sẵn các mạng nội bộ (internal network) và volume lưu trữ. Code chạy trên máy phát triển cá nhân (Local) sẽ có hành vi hoàn toàn giống hệt khi được triển khai lên các máy chủ tính toán đám mây (Cloud), loại bỏ hoàn toàn rủi ro "lỗi do môi trường".
- **Triển khai Local LLM:** Module xử lý AI được tách thành một dịch vụ độc lập trong kiến trúc container, cho phép quản lý tài nguyên (giới hạn CPU, cấp phát RAM/VRAM) một cách tường minh, tránh việc mô hình AI chiếm dụng toàn bộ tài nguyên của các dịch vụ API.

3.1.2. Các công cụ sử dụng

Quá trình xây dựng, kiểm thử và theo dõi hệ thống được thực hiện bằng các công cụ chuyên dụng trong lĩnh vực phát triển phần mềm và quản trị dữ liệu:

- **Visual Studio Code (VS Code):** Môi trường phát triển tích hợp (IDE) chính, hỗ trợ quản lý mã nguồn, viết mã Python (Backend) và TypeScript (Frontend), cùng với việc theo dõi log của Docker container theo thời gian thực.
- **Swagger UI (Tích hợp sẵn qua FastAPI):** Được sử dụng để tài liệu hóa và trực quan hóa các endpoint API. Công cụ này cho phép thực hiện gọi API kiểm thử (API Testing) trực tiếp trên trình duyệt, kiểm tra cấu trúc dữ liệu JSON đầu vào/đầu ra của các module Sinh truy vấn và Đánh giá mà không cần viết script phụ trợ.
- **PgAdmin4:** Nền tảng quản trị cơ sở dữ liệu giao diện đồ họa. Trong đồ án này, PgAdmin4 được dùng để theo dõi, quản lý trực tiếp dữ liệu cấu hình, xem lại các log kết quả đánh giá (evaluation_logs), và giám sát trạng thái của các cơ sở dữ liệu test (như tập Spider, BIRD).

3.1.3. Cấu hình hệ thống

Do hệ thống hoạt động hoàn toàn dựa trên các mô hình ngôn ngữ lớn triển khai cục bộ (Local LLM), việc cấu hình tập trung vào việc định nghĩa các tham số mô hình và siêu tham số xử lý thông qua tệp tin môi trường .env:

- **Loại mô hình (Model Selection):** Biến LLM_MODEL_NAME được thiết lập để trỏ tới các mô hình mã nguồn mở chuyên biệt cho lập trình (như *Qwen-2.5-Coder*, *Llama-3*, hoặc *Mistral*) được host trực tiếp trên máy chủ nội bộ.
- **Tham số sinh văn bản (Generation Parameters):**
 - * **TEMPERATURE:** Được thiết lập ở mức thấp (vd: 0.1 - 0.2) nhằm triệt tiêu tính ngẫu nhiên (hallucination), buộc AI phải sinh ra mã SQL tuân thủ cấu trúc logic chặt chẽ.
 - **MAX_TOKENS:** Giới hạn độ dài chuỗi SQL đầu ra để tránh lãng phí tài nguyên tính toán.

- **Tham số điều phối hàng loạt (Batch Processing):** Cấu hình BATCH_SIZE và MAX_CONCURRENT_REQUESTS để hệ thống biết được năng lực giới hạn của phần cứng (VRAM), từ đó tự động ngắt nhỏ hàng đợi khi chạy đánh giá trên hàng ngàn câu hỏi, ngăn chặn triệt để lỗi tràn bộ nhớ.

3.2. KẾT QUẢ THỰC NGHIỆM

3.2.1. Kịch bản 1: Sinh truy vấn Text-to-SQL cơ bản

- **Mục tiêu:** Kiểm tra độ phản hồi và tính chính xác về mặt cú pháp của module sinh truy vấn (Generator).
- **Đầu vào:** Câu hỏi bằng ngôn ngữ tự nhiên yêu cầu thống kê dữ liệu kết hợp với thông tin schema (bảng, cột, khóa ngoại).
- **Kết quả:** Hệ thống Local LLM đã tiếp nhận prompt và sinh thành công các câu lệnh truy vấn phức hợp bao gồm các mệnh đề SELECT, JOIN (kết nối nhiều bảng), và GROUP BY (nhóm dữ liệu). Tốc độ phản hồi đạt mức tốt (vài giây cho mỗi truy vấn) mặc dù xử lý cục bộ. Đặc biệt, tên các bảng và trường dữ liệu trong câu lệnh SQL hoàn toàn khớp (matching) với cấu trúc schema thực tế đã được cung cấp, không xảy ra hiện tượng "bịa" tên cột.

3.2.2. Kịch bản 2: Tối ưu hoá truy vấn phức tạp

- **Mục tiêu:** Đánh giá năng lực của module Optimizer trong việc đọc hiểu kế hoạch thực thi và đề xuất giải pháp giảm tải chi phí cơ sở dữ liệu.
- **Đầu vào:** Một câu truy vấn SQL lồng nhau kém hiệu quả, gây ra tình trạng quét toàn bảng (Sequential Scan) khiến tổng chi phí thực thi (Total Cost) bị đẩy lên rất cao trên PostgreSQL.
- **Kết quả:** Hệ thống chạy lệnh EXPLAIN (ANALYZE, FORMAT JSON), bóc tách dữ liệu và đưa vào LLM phân tích. Kết quả trả về không chỉ phân tích rõ nguyên nhân gây chậm (nút thắt cổ chai) mà còn đưa ra hai khuyến nghị cụ thể:

1. Viết lại (Rewrite) truy vấn, thay đổi cách lọc dữ liệu ở mệnh đề WHERE kết hợp tối ưu lại cấu trúc JOIN.
2. Đề xuất câu lệnh DDL bổ sung Index (ví dụ: CREATE INDEX idx_user_date ON users (created_at)). Sau khi áp dụng đề xuất, thực nghiệm ghi nhận thời gian quét bảng và Total Cost giảm thiểu đáng kể, tối ưu hóa được hiệu năng truy xuất.

3.2.3. Kịch bản 3: Thực thi Batch trên Benchmark

- **Mục tiêu:** Đánh giá sức chịu tải và tính ổn định của đường ống (Pipeline) Evaluation khi vận hành hàng loạt trên các bộ dữ liệu lớn.
- **Đầu vào:** Tải toàn bộ danh sách hàng ngàn câu hỏi tự nhiên cùng cơ sở dữ liệu tương ứng từ hai bộ tiêu chuẩn Spider và BIRD.
- **Kết quả:** Hệ thống tự động nạp dữ liệu và đẩy vào cơ chế suy luận bất đồng bộ (Asynchronous Inference). Nhờ cấu hình điều phối hàng đợi (Queue Management) tốt, hệ thống duy trì việc gọi Local LLM liên tục mà không bị ngắt quãng. Các tình huống rủi ro như quá tải phần cứng, tràn bộ nhớ đồ họa (OOM) đã được xử lý mượt mà thông qua cơ chế tự động gộp lô (batching) và giải phóng bộ nhớ, minh chứng cho sự ưu việt của việc tự chủ hạ tầng so với việc phụ thuộc vào các API bên ngoài.

3.2.4. Kịch bản 4: Trích xuất Báo cáo Đánh giá (Evaluation Report)

- **Mục tiêu:** Trực quan hóa và phân tích số liệu kết quả đánh giá mô hình.
- **Kết quả:** Sau khi hoàn tất tiến trình Batch Inference, hệ thống giao diện (Dashboard) hiển thị trực quan các biểu đồ số liệu (Metrics). Báo cáo ghi nhận và tính toán chính xác tỷ lệ phần trăm của hai chỉ số cốt lõi: **Execution Accuracy (EX)** và **Exact Match (EM)**.
- **Phân tích lỗi:** Hệ thống không chỉ đưa ra điểm số mà còn liệt kê chi tiết các câu lệnh không vượt qua bài kiểm tra. Các lỗi được phân loại rành mạch

thành lỗi do LLM sinh sai cú pháp (Syntax Error) hoặc lỗi trong quá trình thực thi do sai logic nghiệp vụ (Runtime Error từ PostgreSQL), hỗ trợ đắc lực cho lập trình viên trong việc gỡ lỗi và tinh chỉnh Prompt.

3.2.5. Kịch bản 5: An toàn thực thi với Database Simulation

- **Mục tiêu:** Kiểm thử khả năng chịu lỗi và tính năng bảo vệ an toàn dữ liệu của môi trường Sandbox khi đối mặt với các câu lệnh phá hoại.
- **Đầu vào:** Cố tình chèn vào Prompt hoặc thiết lập nhiệt độ (Temperature) cao để LLM sinh ra các câu lệnh chứa từ khóa nguy hiểm như DROP TABLE users; hoặc các truy vấn lặp vô hạn gây cạn kiệt tài nguyên.
- **Kết quả:** Phân hệ Database Simulation đã hoạt động đúng như thiết kế. Câu lệnh phá hoại được thực thi ngầm nhưng ngay lập tức bị đóng gói trong khối Transaction và kích hoạt cơ chế ROLLBACK. Kết quả ghi nhận trong bảng evaluation_logs là RUNTIME_ERROR, trong khi cấu trúc dữ liệu và các bản ghi của database gốc hoàn toàn không bị ảnh hưởng. Điều này minh chứng cho tính khả thi của hệ thống khi triển khai đánh giá các mô hình AI lạ, chưa được kiểm chứng độ an toàn.

3.3. NHẬN XÉT ĐÁNH GIÁ KẾT QUẢ

Qua các kịch bản thực nghiệm từ thao tác đơn lẻ đến vận hành hàng loạt ở cường độ cao, có thể đưa ra các nhận xét sau về dự án:

1. **Tính ổn định và chính xác:** Hệ thống SQLTuner hoạt động vô cùng ổn định, luồng xử lý từ tiếp nhận yêu cầu, trích xuất schema, suy luận AI cho đến thực thi kiểm thử đều diễn ra theo một logic chặt chẽ và đạt độ chính xác cao.
2. **Khả năng tối ưu hóa thực tiễn:** Module Optimizer đã chứng minh được giá trị thực tế trong việc tự động hóa khâu tinh chỉnh truy vấn, một tác vụ thường tiêu tốn rất nhiều thời gian của các DBA và kỹ sư dữ liệu.

3. **Sự ưu việt của Evaluation Framework:** Bộ công cụ đánh giá tự động (Evaluation Module) hoạt động đúng như thiết kế, làm cơ sở khoa học định lượng (qua chỉ số EX, EM) để minh chứng sự khác biệt về năng lực của các dòng Local LLM khác nhau khi phải đối mặt với các Database Schema lớn, chứa cấu trúc ràng buộc phức tạp. Đây là một bước tiến quan trọng, giúp các nhà phát triển AI và kỹ sư phần mềm có một công cụ tiêu chuẩn, độc lập và bảo mật (hoàn toàn chạy offline) để tinh chỉnh các ứng dụng tương tác cơ sở dữ liệu dựa trên ngôn ngữ tự nhiên.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. KẾT QUẢ ĐẠT ĐƯỢC

Trải qua quá trình tìm hiểu cơ sở lý thuyết, phân tích yêu cầu thực tiễn và tiến hành triển khai kỹ thuật, đề án "**Nghiên cứu và Xây dựng Hệ thống AI Hỗ trợ Sinh, Tối ưu và Đánh giá Truy vấn SQL (SQLTuner)**" đã hoàn thành các mục tiêu đề ra ban đầu. Những đóng góp và kết quả nổi bật của hệ thống bao gồm:

- **Xây dựng thành công kiến trúc hệ thống toàn diện và tối ưu:** Đề án đã phát triển một hệ thống nền tảng vững chắc với kiến trúc phân tầng. Sử dụng framework FastAPI để xử lý bất đồng bộ (asynchronous) mạnh mẽ kết hợp cùng hệ quản trị cơ sở dữ liệu PostgreSQL, hệ thống đảm bảo được hiệu năng cao và khả năng chịu tải tốt. Việc container hóa toàn bộ ứng dụng bằng Docker giúp luồng công việc từ phát triển đến triển khai trở nên nhất quán và dễ dàng bảo trì.
- **Tích hợp liền mạch sức mạnh của Local LLM vào quy trình Kỹ nghệ Dữ liệu:** Thay vì phụ thuộc vào các API bên ngoài, hệ thống đã tích hợp và vận hành thành công các Mô hình ngôn ngữ lớn cục bộ (Local LLM), đảm bảo tuyệt đối tính bảo mật của dữ liệu (Data Privacy) và loại bỏ chi phí gọi API. Hệ thống thực hiện xuất sắc hai tác vụ cốt lõi: (1) Chuyển đổi ngôn ngữ tự nhiên thành truy vấn SQL với độ chính xác cú pháp cao, và (2) Tự động phân tích cây kế hoạch thực thi (Execution Plan) để đưa ra các đề xuất tối ưu hóa mang tính thực tiễn cao cho các kỹ sư dữ liệu.
- **Hoàn thiện phân hệ Đánh giá định lượng (Evaluation Framework):** Đây là đóng góp mang tính học thuật và cốt lõi nhất của đề án. Hệ thống đã chuẩn hóa quy trình kiểm thử các mô hình Text-to-SQL bằng việc nạp và xử lý các bộ dữ liệu benchmark uy tín hàng đầu thế giới (như BIRD, Spider). Thông qua cơ chế thực thi Sandbox tự động, hệ thống đo lường chính xác năng lực của mô hình dựa trên hai chỉ số khắc khe là Độ chính xác thực thi (Execution

Accuracy - EX) và Độ khớp cú pháp (Exact Match - EM). Công cụ này cung cấp một thước đo minh bạch, giúp các nhà phát triển dễ dàng so sánh hiệu năng khi thay đổi phiên bản mô hình hoặc tinh chỉnh cấu trúc Prompt.

2. KIẾN NGHỊ VÀ HƯỚNG PHÁT TRIỂN

Mặc dù đã đạt được những kết quả khả quan, bài toán Text-to-SQL và tối ưu hóa cơ sở dữ liệu vẫn còn nhiều khía cạnh phức tạp cần được giải quyết. Để hoàn thiện và nâng cao giá trị ứng dụng của SQLTuner trong thực tế doanh nghiệp, đề án đề xuất một số hướng nghiên cứu và phát triển tiếp theo như sau:

- **Mở rộng khả năng tương thích đa hệ quản trị cơ sở dữ liệu (Multi-DBMS Support):** Hiện tại, hệ thống đang tập trung sức mạnh vào PostgreSQL. Trong tương lai, cần xây dựng thêm các module Adapter (bộ chuyển đổi phương ngữ SQL) để hệ thống có thể kết nối, sinh truy vấn và phân tích hiệu năng trên nhiều nền tảng khác như hệ quản trị quan hệ truyền thống (MySQL, Oracle, SQL Server) và các kho dữ liệu đám mây (Google BigQuery, Snowflake).
- **Tối ưu hóa hiệu năng tính toán song song (Multiprocessing & Asynchronous Queues):** Khi bộ dữ liệu đánh giá (Benchmark) lên tới hàng chục nghìn câu hỏi, việc chạy Batch Inference trên phần cứng cục bộ sẽ tiêu tốn nhiều thời gian. Hướng phát triển tiếp theo là áp dụng các cơ chế hàng đợi tác vụ phân tán (như Celery, RabbitMQ) và xử lý đa tiến trình (Multiprocessing) để tận dụng tối đa số lượng nhân CPU/GPU, từ đó rút ngắn đáng kể thời gian sinh truy vấn và thực thi đối chiếu tập kết quả.
- **Nghiên cứu áp dụng kỹ thuật RAG (Retrieval-Augmented Generation) cho Schema Linking:** Các Local LLM thường bị giới hạn về cửa sổ ngữ cảnh (Context Window). Đối với các cơ sở dữ liệu quy mô doanh nghiệp (Enterprise ERP/CRM) chứa hàng ngàn bảng và hàng chục ngàn cột, việc đưa toàn bộ Schema vào Prompt là bất khả thi và gây nhiễu thông tin. Việc

áp dụng RAG kết hợp với Cơ sở dữ liệu Vector (Vector Database) sẽ giúp hệ thống trích xuất (retrieve) chỉ những bảng/cột có liên quan trực tiếp đến câu hỏi của người dùng để đưa vào LLM, giúp tăng đột phá độ chính xác của quá trình Schema Linking.

- **Tích hợp cơ chế Fine-tuning mô hình liên tục (Continuous Learning):**

Tận dụng dữ liệu lịch sử từ bảng query_logs và feedbacks (nơi người dùng sửa lỗi các câu lệnh SQL chưa chính xác), có thể xây dựng một luồng dữ liệu tự động để huấn luyện tinh chỉnh (Fine-tuning) Local LLM bằng các kỹ thuật tối ưu tham số hiệu quả như LoRA/QLoRA. Điều này sẽ giúp mô hình ngày càng thông minh hơn và học được các "phương ngữ" nghiệp vụ đặc thù của chính tổ chức đang triển khai hệ thống.

TÀI LIỆU THAM KHẢO

Tiếng Việt [1] Tài liệu giảng dạy chuyên ngành Công nghệ thông tin, Trường Đại học Bách Khoa - Đại học Đà Nẵng.

Tiếng Anh [2] Jinyang Li, Binyuan Hui et al. (2024), *BIRD: A Big Bench for Large-Scale Database Grounded Text-to-SQLs*, NeurIPS. [3] Tao Yu, Rui Zhang et al. (2018), *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*, EMNLP. [4] Sebastián Ramírez (2024), *FastAPI Documentation*, <https://fastapi.tiangolo.com/>.